

LifeAdmin: Progress Report

Kushant Garg (1024030336) Sanvi (1024031191) Loveleen Singh (1024030323)

September 6, 2026

1 Introduction

This report documents the progress made on LifeAdmin, our UCS503P Software Engineering Lab project, since the submission of the initial project proposal. It covers a change in our implementation approach, the technical work completed so far, and the challenges encountered along the way.

2 What We Accomplished This Session

2.1 Technology decision

Our original proposal specified a web-based implementation using Python and Flask. After further discussion, we decided to instead build LifeAdmin as a Java desktop application, using JavaFX for the user interface and JDBC for database connectivity. This decision was made partly to allow the project to also satisfy a parallel Java coursework requirement, and partly because our team wanted hands-on experience with desktop application development and direct database connectivity, which we had not previously worked with. This change in stack does not alter the project's core functionality or goals as described in the original proposal; it only changes the implementation technology.

2.2 Data model and database

We began by designing the central data model for the application: the **Chore** class, which represents a single trackable obligation. Each chore stores a title, a due date, a category (e.g., "License," "Subscription"), and a completion status flag. Alongside this, we set up a MySQL database (using MySQL Workbench for administration) to persist chore data, and implemented a JDBC-based connection layer so that our Java application could communicate with this database using standard SQL operations.

2.3 Core persistence layer

On top of the database connection layer, we implemented a Data Access Object (**ChoreDAO**) responsible for all database interactions related to chores. This includes the following methods, each of which has been individually tested and confirmed to work correctly against the live database:

- **addChore()** – inserts a new chore record into the database.
- **getAllChores()** – retrieves the full list of stored chores, used to populate the dashboard/list view.
- **markDone()** – updates a chore's status to mark it as completed.

- `deleteChore()` – removes a chore record entirely.
- `getChoresDueSoon()` – retrieves only those chores whose due date falls within a specified number of days from the current date, forming the basis of our upcoming reminder feature.

Structuring this logic as a separate DAO class, rather than embedding SQL queries directly inside the user interface code, was a deliberate design choice intended to keep the database logic isolated and easier to maintain and test independently of the UI.

2.4 User interface

With the persistence layer working, we built an initial JavaFX-based user interface. The current version presents a single window containing a text field for entering a chore title, a date picker for selecting the due date, and an “Add Chore” button. Submitting the form calls directly into the `ChoreDAO` layer to insert the new chore into the database, after which the on-screen list view is refreshed to display all currently stored chores. While this is a minimal interface at present, it demonstrates a complete, working pipeline: user input on screen is captured, written to the database, and read back and displayed – validating that all three layers of the application (UI, business logic, and persistence) are correctly connected end to end.

2.5 Version control and collaboration

We created a shared GitHub repository for the project and structured it with a dedicated `documents` folder for proposal and progress-report files, and a `src` folder for Java source code. We configured an appropriate `.gitignore` file to exclude compiler-generated class files and third-party library files from version control, keeping the repository focused on source code and documentation. Work has been committed incrementally, with each commit corresponding to a discrete unit of progress (e.g., initial data model, JDBC connectivity, JavaFX interface), so that our development history is traceable.

3 Problems We Faced

Since this is the first time our team has worked with several of the technologies involved in this project, a significant part of our effort so far has gone into overcoming a learning curve rather than purely feature development. The main difficulties we encountered were:

- **Limited prior exposure to Java:** Our team’s prior programming experience was concentrated in other languages and coursework, so a meaningful portion of our initial effort went into becoming comfortable with core Java syntax and conventions – class structure, access modifiers, static versus instance members, and exception handling – while simultaneously trying to apply these concepts to a real, working project rather than isolated exercises.
- **No prior GUI development experience:** None of us had previously built a graphical user interface. JavaFX’s event-driven programming model – where the interface is described in terms of scenes, stages, and layout containers, and behavior is attached via event handlers rather than executed top-to-bottom – represented a fundamentally different way of thinking about program flow compared to the straightforward, procedural style of code we were used to.
- **No prior experience with backend database connectivity:** Connecting an application directly to a database was also new territory for our team. Understanding how JDBC acts as a bridge between Java code and SQL, and correctly structuring the sequence of opening a connection, preparing and executing a query, and safely processing the results,

took time to internalize – particularly around avoiding resource leaks and handling potential failures gracefully.

- **General syntax and logic-translation issues:** Beyond conceptual learning, we ran into a number of practical coding issues while translating our intended logic into working Java code – including type mismatches, incorrectly structured methods, and figuring out where and how try-catch blocks should wrap our database operations so that errors are handled without crashing the application.
- **Designing an overall application structure from scratch:** Perhaps the broadest challenge was architectural rather than syntactic: deciding how to divide our code into a data model, a database access layer, and a user interface layer, and defining how these layers should communicate with one another. Having no prior experience designing a multi-layered application, we had to consciously work out this separation of concerns rather than defaulting to writing all logic in a single place.

4 Next Steps

Building on this foundation, our next steps are to extend the JavaFX interface with the ability to mark chores as complete and delete them directly from the UI, surface the “due soon” reminder logic visibly on the dashboard, and begin work on the optional document-upload and OCR-based date-detection feature described in our original proposal.